

INTRODUCCION A LA CRIPTOGRAFIA MODERNA

Homework 2

Depto. de Computación, FCEN, UBA, 1er. Bimestre 2026

14/4/26

Student:

Due: 23/4/25, 23:59 hs

Instructions

- Upload your solution to Campus; make sure it's only one file, and clearly write your name on the first page. Name the file '<your last name>_HW2.pdf.'
If you are proficient with $\text{\LaTeX}$, you may also typeset your submission and submit in PDF format. To do so, uncomment the "`%\begin{solution}`" and "`%\end{solution}`" lines and write your solution between those two command lines.
- Your solutions will be graded on *correctness* and *clarity*. You should only submit work that you believe to be correct.
- You may collaborate with others on this problem set. However, you must **write up your own solutions** and **list your collaborators and any external sources (including ChatGPT and similar generative AI chatbots)** for each problem. Be ready to explain your solutions orally to a member of the course's staff if asked.

This assignment contains 5 questions, for a total of 50 points.

1. Let F be a pseudorandom permutation. For each of the following encryption schemes (where the key is a uniform $k \in \{0, 1\}^n$), show how to decrypt and then state whether the scheme is CPA-secure. Explain your answers.
 - (a) (5 points) To encrypt $m \in \{0, 1\}^n$, choose uniform $r \in \{0, 1\}^n$ and output the ciphertext $\langle r, F_k(m) \oplus r \rangle$.

Solution: Para decriptar, tenemos que reconocer los dos elementos de la tupla (asumimos una codificación estandar para esto). Luego, podemos hacer la inversa del XOR con r (el primer elemento de la tupla) para obtener $F_k(m)$. Luego, como tenemos k , aplicamos la inversa de F_k para obtener m .

Vemos que este esquema no es CPA-seguro. Para demostrarlo mostramos una estrategia que puede seguir el adversario en el experimento de Priv_k .

El adversario, una vez que entrega los m_0 y m_1 y recibe la versión encriptada de uno de los dos, pide encriptar al oráculo m_0 . Luego, a ambos ciphertexts los manipula para llegar a $F_k(m_0)$ y $F_k(m_b)$ (lo hace con el primer paso de la decriptación mostrada antes, es trivial para cualquier tupla). De ese modo, como F_k es una función determinística, $F_k(m_0) = F_k(m_b)$ si $b = 0$, y si no son iguales es porque $b = 1$. Luego, el adversario tiene probabilidad mayor a $\frac{1}{2}$ de ganar el experimento y el esquema no es CPA-seguro.

- (b) (5 points) To encrypt $m \in \{0,1\}^n$, choose uniform $r \in \{0,1\}^n$ and output the ciphertext $\langle r, F_r(k) \oplus m \rangle$.

Solution: Para decriptar, tenemos que reconocer los dos elementos de la tupla, r y $F_r(k) \oplus m$. Luego, podemos generar $F_r(k)$ (dado que tenemos k) y hacer la inversa del XOR para obtener m .

Vemos que este esquema tampoco es CPA-seguro. Para demostrarlo veamos el siguiente escenario. El adversario entrega m_0 y m_1 y recibe $c = \langle r, F_r(k) \oplus m_b \rangle$. Luego, conoce r , y F es una función pública. Pide al oráculo que encripte un m_2 tal que $m_2 = 0^n$ y recibe $c' = \langle r_2, F_{r_2}(k) \oplus 0^n \rangle$. Entonces obtiene $F_{r_2}(k)$ haciendo el XOR con 0^n .

Luego, para obtener k puede hacer lo siguiente. Como F es pública, tiene r_2 y tiene $F_{r_2}(k)$, $k = F_{r_2}^{-1}(F_{r_2}(k))$.

Luego, volviendo al ciphertext original, $c = \langle r, F_r(k) \oplus m_b \rangle$, puede obtener m_b haciendo el XOR con $F_r(k)$, dado que ahora conoce k , r y F . Entonces compara m_b con m_0 y m_1 y gana el experimento con probabilidad 1.

2. (10 points) Show that the encryption scheme presented in Lecture 5 (p. 21) is CCA-secure.

Solution: La diferencia entre CCA-seguro y CPA-seguro es que en CCA-seguro el adversario puede decriptar cualquier ciphertext (excepto el target ciphertext).

En este caso, vemos que el adversario recibe un c^* . Las opciones que tiene son:

- Encriptar mensajes arbitrarios.
- Decriptar ciphertexts arbitrarios (posibles modificaciones de c^* , o ciphers de mensajes arbitrarios).

Si encripta mensajes arbitrarios, va a recibir un ciphertext. Desencriptarlo sin modificaciones va a devolver el mismo mensaje, lo que no aporta nueva información. Mas formalmente, si toma m' y pide encriptar tal que $c' = \text{Enc}_k(m')$, pedir decriptar c' va a dar $m' = \text{Dec}_k(c')$.

El cipher en sí es el resultado de una PRP, en el cual la clave es secreta. Entonces no hay mucho que se pueda inferir sobre el contenido de x , r o 0^n a partir de un ciphertext arbitrario.

Si el adversario intenta modificar c^* para luego intentar decriptarlo y obtener información, tiene probabilidades negligibles de encontrar un cipher válido. En particular, de $\frac{1}{2^n}$, dado que los ciphers válidos son los que terminan en 0^n . Además,

si encuentra un cipher que sea válido, con mínimas modificaciones a c^* , no quiere decir que ese cipher modificado tenga el mismo mensaje x .

Por ende, vemos que agregar el oráculo de decripción no aporta nada. Con ver que es CPA-seguro alcanza. Y esto es bastante simple de ver, ya que F_k es PRP, y lo que se encripta recibe siempre un r fresco uniforme de input que hace que las encrpciones sean indistinguibles unas de otras.

3. Prove that the following modifications of basic CBC-MAC **do not** yield a secure MAC (even for fixed-length messages):

- (a) (5 points) Mac outputs all blocks $t_1, \dots, t_\ell$, rather than just t_ℓ . (Verification only checks whether t_ℓ is correct.)

Solution: Para mostrar que el esquema no es seguro, mostramos una estrategia que permite ganar el experimento.

Sea $m = m_1 || m_2 || \dots || m_l$. Pido al oráculo $MAC_k(m) = (t_1, t_2, \dots, t_l)$. Hago lo mismo para $m' = m'_1 || m_2 || \dots || m_l$, y pido al oráculo $MAC_k(m') = (t'_1, \dots, t'_l)$. Notar que $t_1 = F_k(m_1)$ y $t'_1 = F_k(m'_1)$.

Ahora definimos otro mensaje $m^* = m'_1 || t'_1 \oplus t_1 \oplus m_2 || m_3 || \dots || m_l$.

Luego, vamos a construir el $MAC_k(m^*)$ sin pedirlo al oráculo de esta manera.

$$\begin{aligned} t_1^* &= F_k(m'_1) = t'_1 \\ t_2^* &= F_k(t'_1 \oplus t'_1 \oplus t_1 \oplus m_2) = F_k(t_1 \oplus m_2) = t_2 \\ t_3^* &= F_k(t_2 \oplus m_3) = t_3 \end{aligned}$$

Y así sucesivamente hasta completar los l bloques. De ese modo, construimos el tag de un mensaje m^* sin haberlo pedido al oráculo.

- (b) (5 points) A random initial block is used each time a message is authenticated. That is, choose uniform $t_0 \in \{0, 1\}^n$, run basic CBC-MAC over the “message” $t_0, m_1, \dots, m_\ell$, and output the tag $\langle t_0, t_\ell \rangle$. Verification is done in the natural way.

Solution:

Tomamos un mensaje $m = m_1 || \dots || m_l$. Luego pedimos su MAC y conseguimos $\langle t_0, t_l \rangle$.

Notamos que $t_1 = F_k(t_0 \oplus m_1)$ por la construcción del esquema, donde t_0 fue elegido al azar de manera uniforme por el oráculo de MAC.

Luego, definimos un mensaje $m^* = m_1^* || m_2 || m_3 || \dots || m_l$. Le modificamos solo el primer bloque para ver si le podemos generar un tag sin pedirle al oráculo,

solamente con la información de m . Para que sean distintos, necesito $m_1^* \neq m_1$. Veamos que necesitamos pedir lo siguiente:

$$t_1^* = F_k(t_0^* \oplus m_1^*) = t_1 = F_k(t_0 \oplus m_1)$$

Si se cumple esto, el resto de t_i^* van a ser iguales a t_i para cada $i = 2 \dots l$.

Digamos que $m_1^* = x \oplus m_1$, para algún $x \in \{0, 1\}^n$, con $x \neq 0^n$ (así es distinto a m^*). Luego, tenemos que

$$F_k(t_0^* \oplus x \oplus m_1) = F_k(t_0 \oplus m_1)$$

Luego, para anular el x , podemos pedir que $t_0^* = t_0 \oplus x$. De ese modo, $t_1^* = F_k(t_0 \oplus x \oplus x \oplus m_1) = F_k(t_0 \oplus m_1) = t_1$.

Luego, conseguimos t_l aplicando el resto del algoritmo, y t_0^* es $t_0 \oplus x$. De ese modo, podemos generar el tag de m^* sin pedirle al oráculo, es decir que $\text{Vrfy}(\langle t_0^*, t_l^* \rangle, m^*) = 1$.

4. (a) (5 points) Provide formal security definitions for *second preimage resistance* and *preimage resistance* of hash functions, similar to the definition of collision resistance we saw in Lecture 6.
- (b) (5 points) Prove that any hash function that is collision resistant is second-preimage resistant, and any hash function that is second-preimage resistant is preimage resistant.
5. (10 points) Show how to find a collision in the Merkle tree construction if t is not fixed. Specifically, show how to find two sets of inputs $x_1, \dots, x_t$ and $x'_1, \dots, x'_{2t}$ such that $\mathcal{MT}_t(x_1, \dots, x_t) = \mathcal{MT}_{2t}(x'_1, \dots, x'_{2t})$.